

# Éditeur de Mosaïque

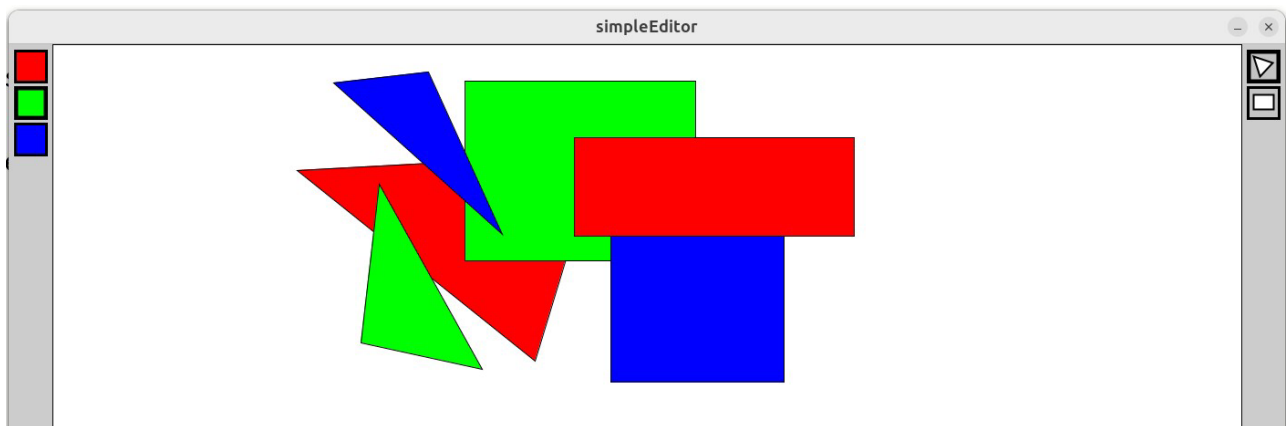
Novembre 2025

## Projet d'info112

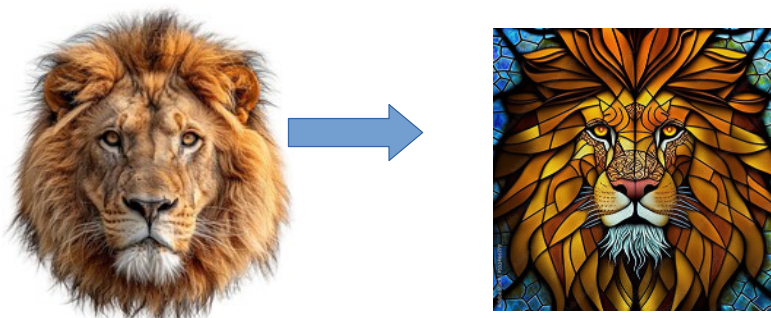
Pour l'informatique graphique à deux dimensions

### Éditeur existant

L'enseignant vous fournit un éditeur minimaliste qui permet de créer des triangles et des rectangles. Le code est organisé en objet afin de simplifier sa compréhension. On ne vous demande pas de modifier cette structure mais de la dupliquer et de la faire évoluer.



L'éditeur permet de sélectionner une couleur parmi trois et une forme parmi deux. Une fois l'outil et la couleur choisie il faut encore cliquer dans la zone centrale de dessin afin de créer la forme souhaitée. Votre première tâche consistera à charger et afficher une photo en fond. Le travail de l'utilisateur consistera à créer des formes de couleur par dessus l'image afin d'en faire une version mosaïque.



## Éditeur de couleur

La première partie de ce projet consiste à faire évoluer la partie gauche afin de permettre à l'utilisateur d'avoir une meilleure palette de couleur. Vous commencerez par remplacer col1, col2 et col3 du code existant par une liste (ArrayList) de couleurs. Vous initialiserez cette liste dans setup() avec plus de couleurs. La fonction draw() devra afficher la liste des couleurs quel qu'en soit le nombre. La fonction mouseClicked() devra permettre de sélectionner n'importe quelle couleur de la liste.

Le point suivant consistera à rajouter un bouton gris à la suite des couleur avec le texte « + » à l'intérieur. Vous créerez également de code dans mouseClicked() qui récupère quand on clique dans ce nouveau bouton et qui changera le mode (ni RECT ni TRIANGLE mais un nouveau mode SELCOLOR. Lorsque l'éditeur sera ainsi en mode SELCOLOR et que l'utilisateur cliquera dans l'image, vous récupérerez la couleur du pixel sous la souris et vous rajouterez cette couleur dans la liste des couleurs. Cet outil fonctionnera comme une pipette de sélection de couleur.

Vous pourrez continuer à faire évoluer votre palette de couleur en permettant la destruction d'une couleur (bouton « - ») ou la modification de la couleur sélectionnée (boutons « plus lumineuse », « moins lumineuse », « plus saturée », etc.)

## Éditeur de formes

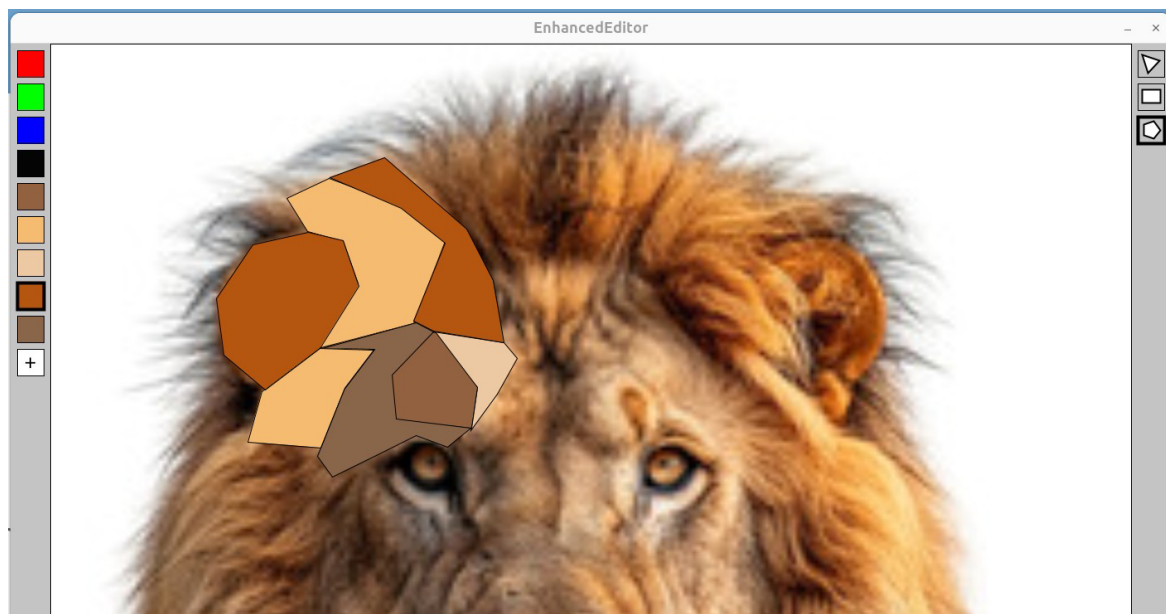
La seconde partie de ce projet consiste à faire évoluer la partie droite afin de permettre à l'utilisateur de créer des formes plus complexes que des triangles et des rectangles.

La forme la plus importante à réaliser est le polygone. Il s'agit d'une généralisation du triangle. Le nouvel objet **Polygon** n'aura pas 2 autres points p2 et p3 comme le triangle mais une liste de points. Toutefois on ne pourra pas compter les points pour savoir si on arrive au dernier point et terminer la forme (et la basculer dans la liste des objets à afficher). Vous utiliserez la distance au premier point comme déclencheur. Lorsque l'utilisateur rajoutera un point très proche (ie <5 pixels du premier points), on terminera la forme.

Vous pourrez créer des formes pre-etablis (une croix, une rosace, un losange, un hexagone, etc.), chacune avec un bouton et une façon spécifique de gérer la création dans les fonctions mousePressed() et mouse Released().

## Éditeur de rendu

La dernière partie consiste à gérer plusieurs mode d'affichage. Par exemple en affichant ou pas l'image de fond, en affichant (ou pas) des bordures aux formes, etc. Ces modes doivent permettre de soutenir au mieux le travail de l'utilisateur. Il doit y avoir un mode qui ressemble le plus possible à un vitrail ou une mosaïque, un mode qui affiche de petits carrés autour de chaque points des formes existantes, etc. Ces mode pourront permettre une meilleure utilisation des couleurs et des formes. Par exemple dans le mode on on affiche de petits carrés autour de chaque points, ces points peuvent agir comme des aimants. Ça signifie que si l'utilisateur essaye de rajouter un point dans un polygone qui est très proche d'un point dans un autre polygone, le nouveau point créé peut s'aligner exactement avec le point existant.



Vous pourrez enfin utiliser une fonction qui teste si la souris ou un point est dans un polygone afin de créer des fonctionnalités supplémentaires (comme changer la couleur d'un polygone existant, déplacer un polygone (mode déplacement), etc.

```
// inspire de Jonas Raoni Soares Silva
// http://jsfromhell.com/math/is-point-in-poly [rev. #0]
boolean isPointIn(PVector pt){
    boolean c = false;
    for(int i = -1, l = pts.size(), j = l - 1; ++i < l; j = i)
        if (((pts.get(i).y <= pt.y && pt.y < pts.get(j).y) || (pts.get(j).y <=
pt.y && pt.y < pts.get(i).y)) && (pt.x < (pts.get(j).x - pts.get(i).x) *
(pt.y - pts.get(i).y) / (pts.get(j).y - pts.get(i).y) + pts.get(i).x))
            c = !c;
    return c;
}
```